

Agile Manifesto Values

1. Individuals and interactions over processes and tools

Traditional limitation:

In traditional models like Waterfall, teams rely heavily on strict processes and documentation. Communication flows through formal reports and approvals, slowing decision-making and reducing collaboration.

How Agile addresses it:

Agile encourages daily communication through stand-ups, direct discussions, and teamwork. Decisions are made faster because people interact directly instead of waiting for process approvals.

2. Working software over comprehensive documentation

Traditional limitation:

Traditional development often spends months creating detailed documents before any working system is available. Stakeholders may not see real progress until late stages.

How Agile addresses it:

Agile focuses on delivering small working features regularly. Stakeholders can test actual software early and give feedback instead of reviewing only documents.

3. Customer collaboration over contract negotiation

Traditional limitation:

Traditional models fix requirements in contracts. Any change requires formal approval and renegotiation, causing delays and conflict.

How Agile addresses it:

Agile involves customers continuously. Feedback is welcomed every sprint, and requirements evolve naturally through collaboration rather

than rigid agreements.

4. Responding to change over following a plan

Traditional limitation:

Traditional projects resist change once planning is completed. Late changes are expensive and discouraged.

How Agile addresses it:

Agile treats change as normal. Backlogs are reprioritized and new requirements are added without disrupting completed work.

Agile Principles — Practical Implications

1. Early and continuous delivery

Practical implication:

Teams release usable features frequently, allowing users to benefit early and provide feedback that improves future work.

2. Welcoming changing requirements

Practical implication:

Developers expect requirement changes and adjust sprint planning instead of resisting updates.

3. Frequent delivery of working software

Practical implication:

Short sprints ensure progress is visible every few weeks, keeping stakeholders confident.

4. Collaboration between business and developers

Practical implication:

Regular meetings ensure business goals and technical solutions stay aligned.

5. Working software as a measure of progress

Practical implication:

Progress is tracked by completed features, not by documents or hours worked.

6. Continuous improvement through reflection

Practical implication:

Teams hold retrospectives to analyze what worked and what didn't, improving future performance.